AI-Assisted Coding

Assignment-9.3

Name: K.Archana

HT No.: 2303A51329

Batch: 20

Task 1: Basic Docstring Generation

1. Python Function with Manual Google-Style Docstring

```
9.3Assignment.py > ...
def sum_even_odd(numbers):
    even_sum=0
    odd_sum=0
    for num in numbers:
        if num%2==0:
            even_sum+=num
        else:
            odd_sum+=num
    return even_sum,odd_sum

result=sum_even_odd([1,2,3,4,5,6])
print(result)
```

```
(12, 9)
(12, 9)
```

2. AI-Generated Docstring
   Prompt: Add a Google-style docstring to this function without modifying the code logic.

```
def sum_even_odd(numbers):
    """Calculates the sum of even and odd numbers in a list.
    This function takes a list of numbers and separates them into two categories:
    even numbers and odd numbers. It then returns the sum of each category.
    Args:
        numbers (list): A list of integers to be processed.
    Returns:
        tuple: A tuple containing two integers (even_sum, odd_sum) where:
            - even_sum (int): The sum of all even numbers in the input list.
            - odd_sum (int): The sum of all odd numbers in the input list.
    Example:
        >>> sum_even_odd([1, 2, 3, 4, 5, 6])
        (12, 9)
        The even numbers are [2, 4, 6] with sum = 12
        The odd numbers are [1, 3, 5] with sum = 9
    """
    even_sum=0
    odd_sum=0
    for num in numbers:
        if num%2==0:
            even_sum+=num
        else:
            odd_sum+=num
    return even_sum,odd_sum

result=sum_even_odd([1,2,3,4,5,6])
print(result)
```

```
(12, 9)
```

Observation:

During this experiment, a Python function was written to calculate the sum of even and odd numbers in a list. A manual Google-style docstring was added to the function, and an AI-generated docstring was also produced using an AI-assisted coding tool.It was observed that the manually written docstring was more detailed and included clear descriptions, structured formatting, and example usage. The AI-generated docstring was correct and readable but shorter and sometimes missing examples or detailed explanations.This experiment showed that AI tools can quickly generate documentation, which helps save time. However, manual review and refinement are necessary to ensure clarity, completeness, and accuracy in documentation.

Task 2: Automatic Inline Comments

1. Manually Written Inline Comments

```
#Task2
class sru_student:
    # constructor to initialize student details
    def __init__(self,name,roll_no,hostel_status):
        self.name=name  # store student name
        self.roll_no=roll_no  # store roll number
        self.hostel_status=hostel_status  # store hostel status (Yes/No)
    # method to update fee
    def fee_update(self,amount):
        self.fee=amount  # assign fee amount
        print("Fee updated successfully")  # confirmation message
    # method to display student details
    def display_details(self):
        print("Name:",self.name)  # print name
        print("Roll No:",self.roll_no)  # print roll number
        print("Hostel Status:",self.hostel_status)  # print hostel status
        if hasattr(self,'fee'):  # check if fee exists
            print("Fee:",self.fee)  # print fee amount
student1=sru_student("Nandhitha","123","Yes")
student1.fee_update(5000)
student1.display_details()
```

```
Fee updated successfully
Name: Nandhitha
Roll No: 123
Hostel Status: Yes
Fee: 5000
```

2.AI-Generated Inline Comments

Prompt:

Add meaningful inline comments to this Python class without modifying the code.

```python
#Task2
class sru_student:
    # constructor to initialize student details
    def __init__(self,name,roll_no,hostel_status):
        self.name=name  # store student name
        self.roll_no=roll_no  # store roll number
        self.hostel_status=hostel_status  # store hostel status (Yes/No)
    # method to update fee
    def fee_update(self,amount):
        self.fee=amount  # assign fee amount
        print("Fee updated successfully")  # confirmation message
    # method to display student details
    def display_details(self):
        print("Name:",self.name)  # print name
        print("Roll No:",self.roll_no)  # print roll number
        print("Hostel Status:",self.hostel_status)  # print hostel status
        if hasattr(self,'fee'):  # check if fee exists
```

Generate code

Add Context...

Auto

```python
            print("Fee:",self.fee)  # print fee amount
            print("Fee:", self.fee)  # print fee amount if it exists
```

Keep Undo

```python
student1=sru_student("Nandhitha","123","Yes")
student1.fee_update(5000)
student1.display_details()
```

```
Fee updated successfully
Name: Nandhitha
Roll No: 123
Hostel Status: Yes
Fee: 5000
```

Observation

In this experiment, a Python class for managing student details was implemented. Inline comments were first written manually to explain each part of the code. Then, an AI-assisted tool was used to automatically generate inline comments for the same program.It was observed that AI-generated comments were clear and helped in understanding the basic functionality of the code. However, some comments were generic and less detailed compared to manually written comments. Manual comments were more precise and better explained the logic of the program.This experiment showed that AI tools can quickly generate useful inline comments, but developers should review them to ensure accuracy, clarity, and completeness.

Task 3: Module-Level and Function-Level Documentation

1. Python Script with Manual NumPy-Style Docstrings

```python
#task3
"""
Calculator Module

This module provides basic arithmetic operations such as
addition, subtraction, multiplication, and division.
"""
def add(a,b):
    """
    Add two numbers.
    """
    return a+b
def subtract(a,b):
    """
    Subtract two numbers.
    """
    return a-b
def multiply(a,b):
    """
    Multiply two numbers.
    """
    return a*b
def divide(a,b):
    """
    Divide two numbers.
    """
    if b==0:
        return "Division by zero not allowed"
    return a/b
print("Addition:",add(10,5))
print("Subtraction:",subtract(10,5))
print("Multiplication:",multiply(10,5))
print("Division:",divide(10,5))
```

```
Multiplication: 50
Addition: 15
Subtraction: 5
Multiplication: 50
Multiplication: 50
Division: 2.0
```

2.AI-Generated Module and Function Documentation

Prompt:Add a module-level docstring and function-level docstrings to this Python calculator code without changing the logic.

```python
#task3
"""
Calculator Module

This module provides basic arithmetic operations such as
addition, subtraction, multiplication, and division.
"""
def add(a,b):
    """
    Add two numbers.
    """
    return a+b
def subtract(a,b):
    """
    Subtract two numbers.
    """
    return a-b
def multiply(a,b):
    """
    Multiply two numbers.
    """
    return a*b
def divide(a,b):
    """
    Divide two numbers.
    """
    if b==0:
        return "Division by zero not allowed"
    return a/b
print("Addition:",add(10,5))
print("Subtraction:",subtract(10,5))
print("Multiplication:",multiply(10,5))
print("Division:",divide(10,5))
```

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
```

Observation

In this experiment, a calculator module containing multiple functions such as addition, subtraction, multiplication, and division was developed. Manual NumPy-style docstrings were written for each function to provide structured documentation. Then, AI-assisted tools were used to generate module-level and function-level documentation automatically.It was observed that the manually written documentation was more detailed and followed a clear NumPy-style structure with proper parameter and return descriptions. The AI-generated documentation was correct and readable but shorter and less detailed in some sections.This experiment demonstrated that AI tools can quickly generate documentation for multi-function scripts, which helps improve development speed. However, manual review and refinement are necessary to ensure completeness, accuracy, and proper structure in professional documentation.